Module 3 Tangent: Syntax

Syntax Sugar

A little bit of sweetness makes the cherry on top.

0.

Assignment Review

Module 3 Quiz: Parsing

1.

Syntax vs Semantics

AKA Appearance vs Personality

Syntax vs Semantics

Syntax

Syntax describes the textual structure of a language (in effect, the grammar).

- A variable declaration looks like:
  - `LET identifier = expr;`
- Good syntax helps quickly identify meaning based on the text alone.
  - Is `name = expr;` a variable declaration or assignment?

Semantics

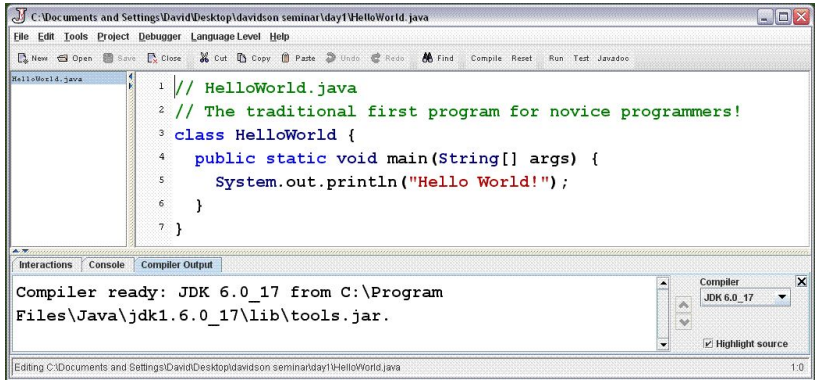
Semantics describe the actual meaning of code independent from how it is written.

- A variable declaration actually means:
  - Define “identifier” to have the value “expr”.
- Semantics cover all of the critical behavior:
  - What restrictions beyond syntax apply?
    - what if “identifier” is already defined?
  - What is the actual runtime behavior?
    - evaluate “expr”, assign to “identifier”.
- A language’s “power” comes from semantics.

Syntax “Categories”

Categories

- Structural code: Code that defines structure but isn’t really evaluated.
- Executional code: Code that is evaluated at runtime.
  - Statements: Code that performs a side effect for computation.
  - Expressions: Code that evaluates to a value (maybe side effect).



Why does Syntax matter?

1. Syntax is the *visual* representation of semantics.
2. Readability helps understanding and reduces mistakes.

```
static OSStatus
SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa, SSLBuffer signedParams,
                                 uint8_t *signature, UInt16 signatureLen)
{
    OSStatus        err;
    SSLBuffer       hashOut, hashCtx, clientRandom, serverRandom;
    uint8_t         hashes[SSL_SHA1_DIGEST_LEN + SSL_MD5_DIGEST_LEN];
    SSLBuffer       signedHashes;
    uint8_t         *dataToSign;
    size_t          dataToSignLen;

    ...
    if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
        goto ↓fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
        goto ↓fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto ↓fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto ↓fail;
        goto ↓fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto ↓fail;

    err = sslRawVerify(ctx,
                       ctx->peerPubKey,
                       dataToSign,              /* plaintext */
                       dataToSignLen,           /* plaintext length */
                       signature,
                       signatureLen);
    if(err) {
        sslErrorLog("SSLDecodeSignedServerKeyExchange: sslRawVerify "
                    "returned %d\n", (int)err);
        goto ↓fail;
    }
```

Apple SSL “goto fail” Bug

```java
public class Main {
  public static void main(String[] args) {
    for (int i = 1; i <= 100; i++) {
      if (i % 3 == 0 && i % 5 == 0) {
        System.out.println("FizzBuzz");
      } else if (i % 3 == 0) {
        System.out.println("Fizz");
      } else if (i % 5 == 0) {
        System.out.println("Buzz");
      } else {
        System.out.println(i);
      }
    }
  }
}
```

Java: Blocks via "{" & "}"

```ruby
class Main
  def main(args)
    for i in 1..100 do #optional???
      if i % 3 == 0 && i % 5 == 0
        puts "FizzBuzz"
      elsif i % 3 == 0
        puts "Fizz"
      elsif i % 5 == 0
        puts "Buzz"
      else
        puts i
      end
    end
  end
end
```

Ruby: Blocks via "do" (sometimes) & "end"

```python
class Main:
  def main(args):
    for i in range(1, 101):
      if i % 3 == 0 and i % 5 == 0:
        print("FizzBuzz")
      elif i % 3 == 0:
        print("Fizz")
      elif i % 5 == 0:
        print("Buzz")
      else:
        print(i)
```

Python: Blocks via indentation (sort of)

Design Review: Blocks

1. Syntax is the *visual* representation of semantics.
2. Readability helps understanding and reduces mistakes.

2.

Syntax Sugar

<insert Maroon 5 joke here>

Syntax Sugar

Definition

Syntax sugar is syntax that simplifies a common action in a language.

- Doesn't change the overall "power" (functionality) of a language*!
  - Sugar can be "lowered" to other, more basic language features.
- Syntax sugar can help or hurt readability - is meaning clearer?
  - Frequency of use helps a lot.

```
//var "desugars" to the value type
var name = "value";
String name = "value";

//for an arbitrary value:
var name = value;
typeof(value) name = value;
```

Why does Syntax matter?

1. Syntax is the *visual* representation of semantics.
2. Readability helps understanding and reduces mistakes.

3.

Sugar; Not Sugar!

The new game show that's sweeping the nation!

```java
var name = "string";

var name = value;
```

```java
String name = "string";

typeof(value) name = value;
```

Syntax Sugar!

Java: Var Type Inference

```java
for (int i = 0; i < 10; i++) {
  System.out.println(i);
}
```

```java
int i = 0;
while (i < 10) {
  System.out.println(i);
  i++;
}

{
  //i must be in a new scope!
  int i = 0;
  while (i < 10) {
    System.out.println(i);
    i++;
  }
}
```

Technically Syntax Sugar!

Java: For Loop

```java
var list = List.of(1, 2, 3);

for (var element : list) {
  System.out.println(element);
}
```

```java
var list = List.of(1, 2, 3);

for (
  var iterator = list.iterator();
  iterator.hasNext();
) {
  //make sure next is called once!
  var element = iterator.next();
  System.out.println(element);
}
```

Syntax Sugar!

Java: For-Each (Enhanced) Loop

```java
var list= List.of(1, 2, 3);

list.forEach(element -> {
  System.out.println(element);
});
```

```java
var list= List.of(1, 2, 3);

list.forEach(new Consumer<>() {
  @Override
  public void accept(Integer elem) {
     System.out.println(elem);
  }
});
```

Syntax Sugar!

Java: Lambda Expression Block

```java
var list= List.of(1, 2, 3);

list.forEach(
  elem -> System.out.println(elem)
);
```

```java
var nums = List.of(1, 2, 3);

list.forEach(element -> {
  System.out.println(element);
});
```

Syntax Sugar!

Java: Lambda Expression Expr

```java
var list = List.of(1, 2, 3);

list.forEach(
  System.out::println
);
```

```java
var list = List.of(1, 2, 3);

list.forEach(
  elem -> System.out.println(elem)
);
```

Syntax Sugar!

Java: Method Reference

```java
public record Vector(
  int x,
  int y
) {}
```

```java
public final class Vector {
  private final int x;
  private final int y;

  public Vector(int x, int y) { … }

  // "fluent" getters: x() = getX()
  public int x() { … }
  public int y() { … }

  public boolean equals(Object other) { … }
  public int hashCode() { … }
  public String toString() { … }
}
```

NOT Syntax Sugar!

Java: Records

Module 3 Tangent: Syntax

Syntax Sugar

A little bit of sweetness makes the cherry on top.

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals